CS 623 Final Project Report: Smart EV Charging Reservation Platform with AWS Cloud Integration

Team Members: Inderpal Singh
Date: June 5, 2026

CS 623 Cloud Computing Final Project Report

Smart EV Charging Reservation Platform with AWS Cloud Integration

Student: Inderpal Singh

1. Project Description

The Smart EV Charging Reservation Platform is a cloud-based application designed to allow users to create and manage electric vehicle charging reservations through a web-based interface. The objective of the project was to demonstrate the integration of frontend technologies with cloud computing services while providing a simple reservation workflow for EV charging stations.

The system utilizes a React frontend for user interaction, AWS API Gateway for request handling, AWS Lambda for serverless backend processing, and Amazon DynamoDB for cloud-based data storage. The project demonstrates how user inputs entered through a web application can be processed by cloud services and stored within a managed database environment.

The final implementation focuses on reservation creation, cloud service interaction, and end-to-end data flow between the user interface and AWS infrastructure.

2. Functionality

The platform provides the following functionality:

• Display available EV charging stations.

• Allow users to enter reservation information including:

- Customer Name
- Vehicle Information
- Charging Station Selection
- Time Slot

• Submit reservation requests through the web application.

• Transfer reservation data to AWS API Gateway.

• Process reservation requests using AWS Lambda.

• Store reservation records in Amazon DynamoDB.

• Display submitted reservation information within the frontend interface.

The application demonstrates a complete cloud-based reservation workflow from user input to cloud storage.

3. System Workflow

The workflow of the system is shown below:

1. User opens the React web application.
2. User enters reservation information.
3. User clicks the Create Reservation button.
4. React sends an HTTP POST request to AWS API Gateway.
5. API Gateway invokes the AWS Lambda function.
6. Lambda processes the request data.
7. Lambda generates a unique reservation identifier.
8. Lambda stores the reservation record in Amazon DynamoDB.
9. DynamoDB persists the reservation information.
10. Lambda returns a success response.
11. The frontend displays confirmation of the reservation submission.

This workflow demonstrates communication between frontend technologies and cloud services using a serverless architecture.

4. Frontend, Backend, and Cloud Services

Frontend

The frontend was developed using React.

The user interface allows users to:

• Enter reservation details.

• Select charging stations.

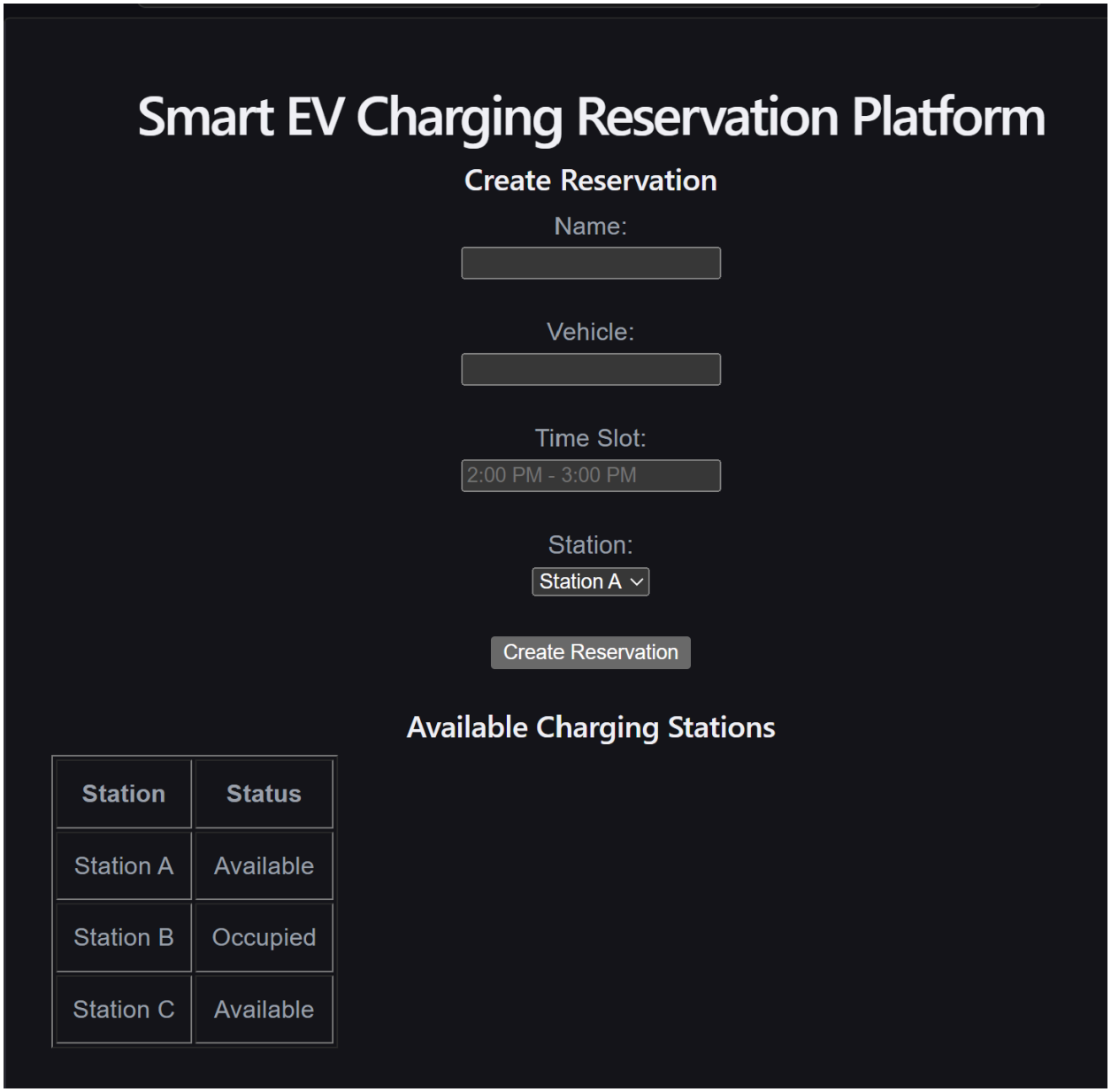
• Submit reservation requests.

• View reservation information after submission.

The frontend communicates directly with AWS API Gateway using HTTP requests.

Figure 1. React Frontend Reservation Interface.

The Smart EV Charging Reservation Platform frontend developed using React. Users can enter reservation information including customer name, vehicle information, charging station selection, and reservation time slot before submitting the request to AWS cloud services.



AWS API Gateway

AWS API Gateway serves as the entry point for all frontend requests.

Responsibilities include:

• Receiving HTTP requests from the React application.

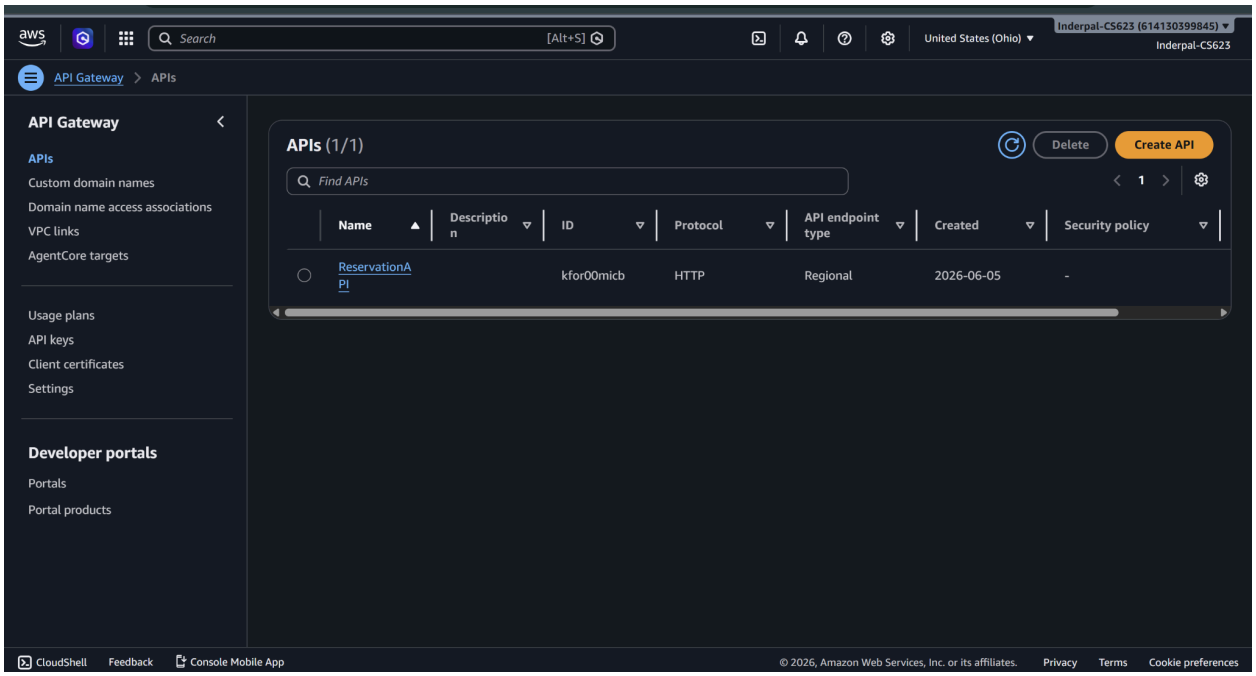
• Routing requests to AWS Lambda.

• Providing an API endpoint accessible by the frontend.

The API Gateway acts as the communication layer between the user interface and cloud infrastructure.

Figure 2. AWS API Gateway Configuration for ReservationAPI.

The API Gateway provides the public HTTP endpoint used by the React frontend to submit reservation requests to the cloud backend.



AWS Lambda

AWS Lambda provides the serverless backend processing component.

Responsibilities include:

• Receiving reservation requests.

• Parsing user input.

• Generating reservation identifiers.

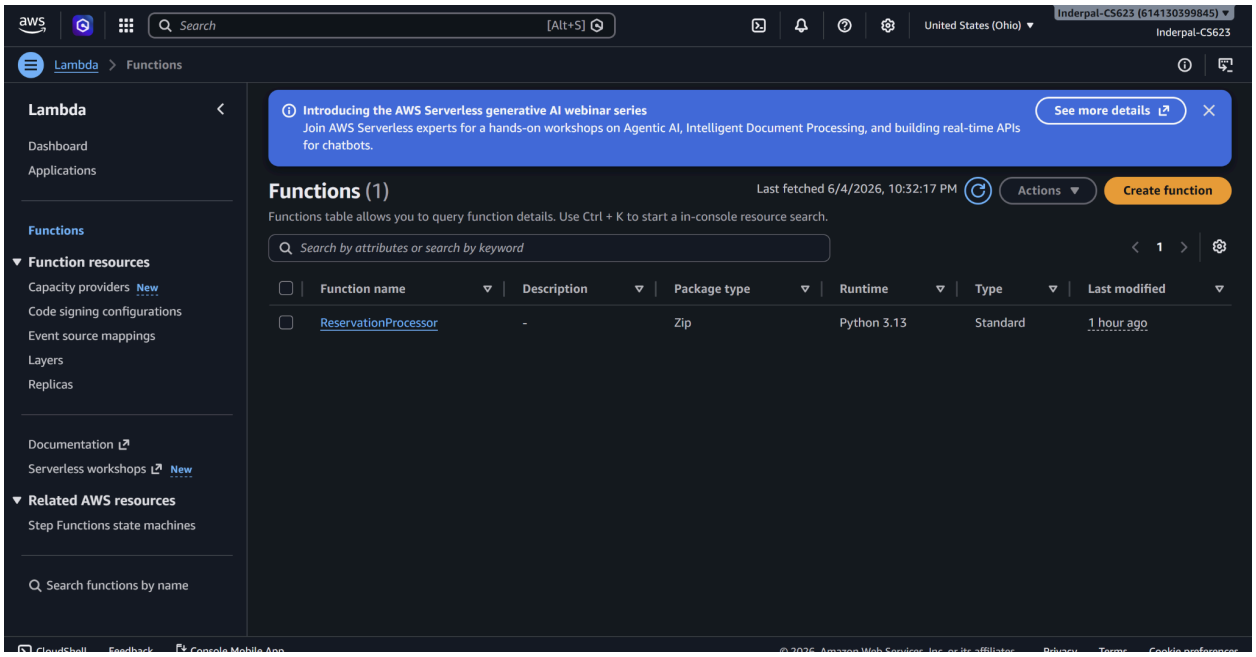
• Formatting reservation data.

• Writing reservation records to DynamoDB.

Lambda eliminates the need to manage dedicated backend servers.

Figure 3. AWS Lambda Function (ReservationProcessor).

The ReservationProcessor AWS Lambda function provides the serverless backend logic for the application. It receives reservation requests from API Gateway, processes the submitted reservation information, generates a unique reservation identifier, and stores the reservation record in the DynamoDB Reservations table.



Amazon DynamoDB

Amazon DynamoDB serves as the cloud database.

Responsibilities include:

• Storing reservation information.

• Managing reservation records.

• Providing scalable cloud-based storage.

The following information is stored for each reservation:

• Reservation ID

• Customer Name

• Vehicle

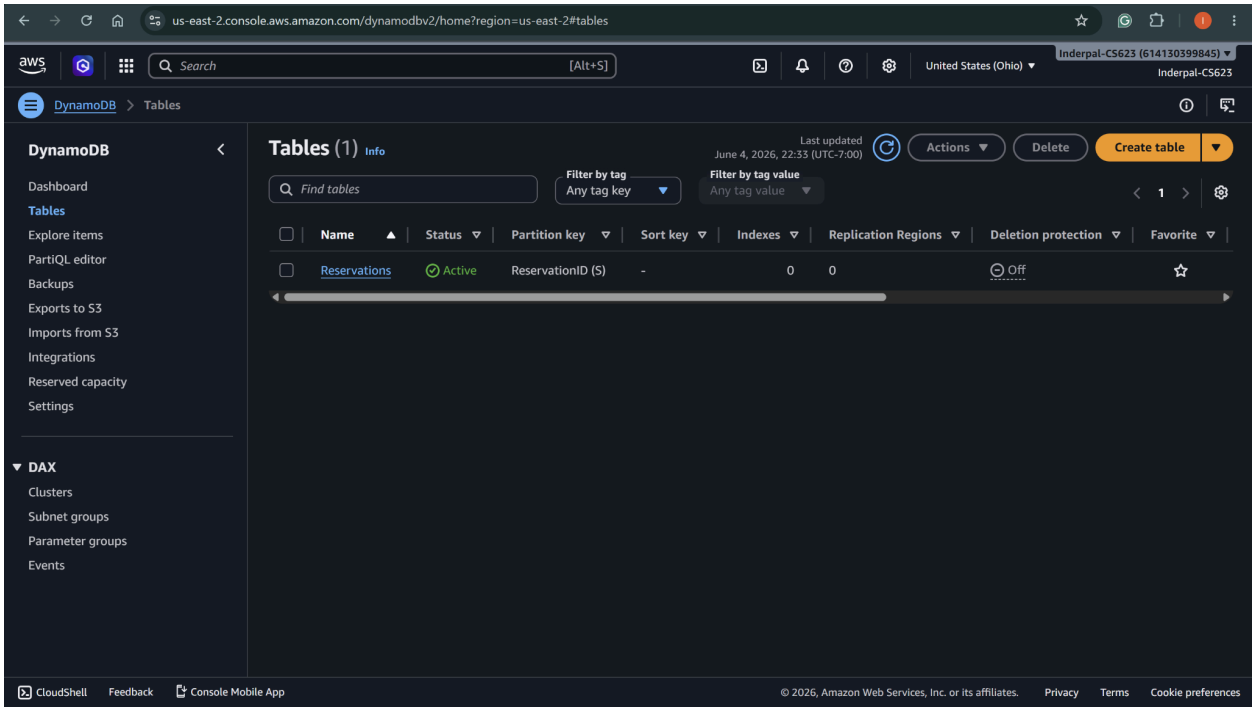
• Station

• Time Slot

• Creation Timestamp

Figure 4. AWS DynamoDB Reservations Table.

The DynamoDB Reservations table serves as the cloud database for the Smart EV Charging Reservation Platform. Reservation records generated through the frontend application are stored in this table using ReservationID as the primary key.



5. Demonstration Results

The completed implementation successfully demonstrated frontend-to-cloud integration.

The following functionality was verified:

• React frontend successfully submitted reservation requests.

• AWS API Gateway successfully received frontend requests.

• AWS Lambda successfully processed reservation data.

• Amazon DynamoDB successfully stored reservation records.

• Reservation information appeared correctly in the cloud database.

Example reservation data successfully stored in DynamoDB included:

Name: Inder

Vehicle: Tesla

Station: Station A

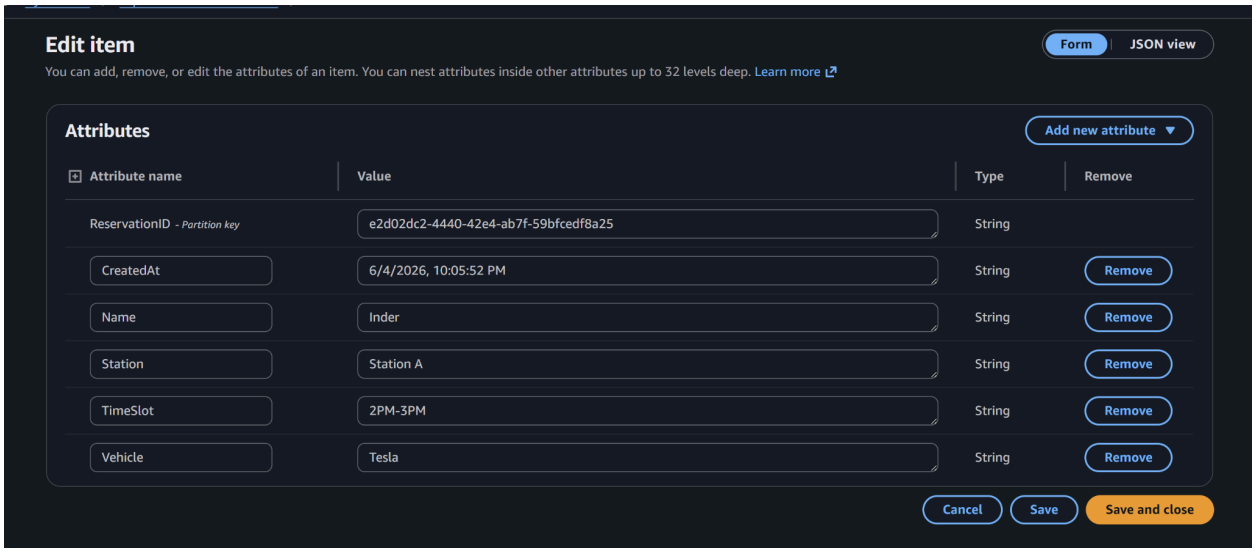
Time Slot: 2PM-3PM

Created At: 6/4/2026 10:05:52 PM

The successful storage of reservation information confirmed that the complete cloud architecture was functioning correctly.

Figure 5. Reservation Record Successfully Stored in DynamoDB.

This reservation record was created through the React frontend application after a user submitted a reservation request. The request was transmitted through AWS API Gateway, processed by the ReservationProcessor AWS Lambda function, and stored in the DynamoDB Reservations table. The successful storage of this record demonstrates complete frontend-to-cloud integration and cloud-based data persistence.



6. Project Evaluation

The project successfully met its primary objective of demonstrating cloud service integration within a reservation management application.

Major accomplishments include:

• Development of a React frontend.

• Deployment of AWS API Gateway.

• Deployment of AWS Lambda.

• Deployment of Amazon DynamoDB.

• Successful frontend-to-cloud communication.

• Successful storage of reservation data in a cloud database.

The implementation demonstrates a practical serverless architecture suitable for reservation-based applications.

7. Cloud Service Justification

The selected AWS services were chosen because they align with modern cloud-native application architectures.

AWS API Gateway was selected because it provides a managed interface for receiving frontend requests.

AWS Lambda was selected because it enables serverless backend execution without managing dedicated servers.

Amazon DynamoDB was selected because it provides scalable and highly available cloud database storage.

Together, these services create a lightweight serverless architecture capable of processing reservation transactions efficiently.

Potential future enhancements include:

• AWS Cognito for user authentication and authorization.

• Stripe payment integration for reservation payments.

• Reservation modification and cancellation features.

• Administrative dashboards for charger management.

These features were identified as future enhancements beyond the scope of the final implementation.

8. Conclusion

This project demonstrated the successful implementation of a cloud-integrated EV charging reservation platform using AWS services.

The final system provides an end-to-end workflow in which reservation information entered through a React frontend is processed through AWS API Gateway and AWS Lambda before being stored in Amazon DynamoDB.

The project successfully demonstrates cloud service deployment, frontend integration, serverless processing, and cloud-based data persistence, fulfilling the objectives of the project.

9. Installation Guide

Requirements:

• Node.js

• React

• AWS Account

• AWS Lambda

• AWS API Gateway

• Amazon DynamoDB

Frontend Setup:

1. Open project directory.
2. Install dependencies:

npm install

3. Start application:

npm run dev

4. Open browser:

http://localhost:5173

AWS Setup:

1. Create DynamoDB table named Reservations.
2. Create Lambda function ReservationProcessor.
3. Configure Lambda permissions for DynamoDB access.
4. Create API Gateway endpoint.
5. Connect API Gateway to Lambda.
6. Update frontend API endpoint URL.
7. Deploy application.

10. Architecture Diagram

User

↓

React Frontend

↓

AWS API Gateway

↓

AWS Lambda

↓

Amazon DynamoDB

This architecture demonstrates a serverless cloud computing solution for EV charging reservation management.